Московский Авиационный Институт
(Национальный Исследовательский Университет)

Институт №8 "Компьютерные науки и прикладная математика"
Кафедра 806 вычислительной математики и программирования

Лабораторная работа №4 по курсу
«Операционные системы»

ДИНАМИЧЕСКИЕ БИБЛИОТЕКИ

Студент: Рязанкина Александра Олеговна
Группа: М8О–203БВ–24
Вариант: 6
Преподаватель: Соколов Андрей Алексеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2026.

Постановка задачи

Цель работы
Целью является приобретение практических навыков в:
- Создание динамических библиотек
- Создание программ, которые используют функции динамических библиотек

Задание
Требуется создать динамические библиотеки, которые реализуют заданный вариантом функционал. Далее использовать данные библиотеки 2-мя способами:
1. Во время компиляции (на этапе «линковки»/linking)
2. Во время исполнения программы. Библиотеки загружаются в память с помощью интерфейса ОС для работы с динамическими библиотеками

В конечном итоге, в лабораторной работе необходимо получить следующие части:
- Динамические библиотеки, реализующие контракты, которые заданы вариантом;
- Тестовая программа (программа №1), которая используют одну из библиотек, используя информацию полученные на этапе компиляции;
- Тестовая программа (программа №2), которая загружает библиотеки, используя только их относительные пути и контракты.

Провести анализ двух типов использования библиотек.

Вариант 6

Функции:
1. SinIntegral(float A, float B, float e) — Рассчет интеграла функции sin(x) на отрезке [A, B] с шагом e.
2. Square (float A, float B) — Подсчет площади плоской геометрической фигуры по двум сторонам.

Реализации:
- Реализация 1:
  - SinIntegral: Метод прямоугольников.
  - Square: площадь прямоугольника.
- Реализация 2:
  - SinIntegral: Метод трапеций.
  - Square: площадь прямоугольного треугольника.

Общие сведения о программе

Функции для работы с динамическими библиотеками (интерфейс ОС):

1. **dlopen** — загружает динамическую библиотеку (.so файл) в адресное пространство процесса. Возвращает «хэндл» (указатель), который используется для дальнейшей работы с этой библиотекой.
2. **dlsym** — ищет адрес функции по её имени внутри уже загруженной библиотеки. Позволяет программе вызвать функцию, о которой она «не знала» на этапе компиляции.
3. **dlclose** — информирует систему о том, что библиотека больше не нужна. Если это было последнее обращение к библиотеке, она выгружается из памяти.
4. **dlerror** — возвращает текстовое описание последней ошибки, произошедшей при вызове функций dlopen, dlsym или dlclose.

Функции стандартной библиотеки C для организации ввода/вывода:

1. **fgets** — безопасно считывает строку текста из стандартного ввода (stdin). В отличие от gets, предотвращает переполнение буфера, так как принимает его размер.
2. **sscanf** — анализирует считанную строку и извлекает из неё данные по заданному формату (например, числа типа float). Это позволяет гибко обрабатывать ввод команд с разным количеством аргументов.
3. **strcspn** — используется для поиска символа новой строки \n в конце введенной пользователем строки, чтобы корректно «обрезать» её перед обработкой.
4. **printf** — выводит форматированные данные на экран (в стандартный поток вывода stdout).

Математическая библиотека:

1. **sinf** — вычисляет значение синуса для числа с плавающей точкой (float).

Общий метод и алгоритм решения.

Для реализации поставленной задачи (создание и использование динамических библиотек) необходимо:

1. Изучить теоретическую базу: освоить принципы динамической линковки (link-time) и динамической загрузки (runtime), работу с заголовочными файлами.

2. Определить общий контракт (contract.h): описать единый интерфейс функций (прототипы для вычисления интеграла sin(x) и площади фигуры), который обеспечит совместимость обеих библиотек с тестовыми программами.
3. Разработать две реализации библиотек (lib1.c и lib2.c): в первой библиотеке реализовать алгоритмы метода прямоугольников для интеграла и формулу площади прямоугольника; во второй библиотеке реализовать метод трапеций для интеграла и формулу площади прямоугольного треугольника; обеспечить идентичность имен функций в обоих файлах для корректного поиска символов.
4. Создать программу №1 (статическая линковка): реализовать пользовательский интерфейс для вызова функций напрямую. Настроить систему сборки так, чтобы исполняемый файл связывался с одной из библиотек на этапе компиляции, а поиск символов происходил автоматически при запуске.
5. Создать программу №2 (динамическая загрузка): использовать интерфейс ОС (dlopen, dlsym, dlclose) для работы с библиотеками как с объектами в памяти; реализовать механизм переключения реализаций по команде «0», обеспечив корректное закрытие текущей библиотеки и загрузку новой по относительному пути без прерывания работы программы.
6. Организовать обработку ввода: в обеих программах реализовать парсинг командной строки для выполнения функций с аргументами и вывода результатов в консоль, предусмотрев обработку некорректного ввода.
7. Провести анализ: сравнить оба подхода, выделив преимущества динамической загрузки (гибкость, подмена логики "на лету") и статической линковки (простота реализации, скорость вызова).

Основные файлы программы

contract.h:

```
#ifndef CONTRACT_H
#define CONTRACT_H

// интеграл sin(x)
float SinIntegral(float A, float B, float e);

// площадь фигуры
float Square(float A, float B);

#endif
```

lib1.c:

```
#include <math.h>
#include <stdio.h>
#include "../include/contract.h"

// метод прямоугольников для интеграла sin(x)
float SinIntegral(float A, float B, float e)
{
    if (e <= 0.0f)
    {
        printf("[lib1] Ошибка: шаг должен быть положительным\n");
        return 0.0f;
    }

    // если A > B, меняем местами
    if (A > B)
    {
        float temp = A;
        A = B;
        B = temp;
    }

    float integral = 0.0f;
    float x = A;
    int steps = (int)((B - A) / e);

    // метод средних прямоугольников
    for (int i = 0; i < steps; i++)
    {
        float midpoint = x + e / 2.0f;
        integral += sin(midpoint) * e;
        x += e;
    }

    // обработка остатка
    if (x < B)
    {
        float midpoint = x + (B - x) / 2.0f;
        integral += sin(midpoint) * (B - x);
    }

    printf("[lib1] Метод прямоугольников, шаг: %.6f\n", e);
    return integral;
}

// площадь прямоугольника
float Square(float A, float B)
{
```

```
    if (A <= 0.0f || B <= 0.0f)
    {
        printf("[lib1] Ошибка: стороны должны быть положительными\n");
        return 0.0f;
    }

    printf("[lib1] Фигура: прямоугольник\n");
    return A * B;
}
```

lib2.c:

```
#include <math.h>
#include <stdio.h>
#include "../include/contract.h"

// метод трапеций для интеграла sin(x)
float SinIntegral(float A, float B, float e)
{
    if (e <= 0.0f)
    {
        printf("[lib2] Ошибка: шаг должен быть положительным\n");
        return 0.0f;
    }

    // если A > B, меняем местами
    if (A > B)
    {
        float temp = A;
        A = B;
        B = temp;
    }

    float integral = 0.0f;
    float x = A;
    int steps = (int)((B - A) / e);

    // метод трапеций
    for (int i = 0; i < steps; i++)
    {
        float left = sin(x);
        float right = sin(x + e);
        integral += (left + right) * e / 2.0f;
        x += e;
    }

    // обработка остатка
    if (x < B)
    {
        float left = sin(x);
        float right = sin(B);
        integral += (left + right) * (B - x) / 2.0f;
    }

    printf("[lib2] Метод трапеций, шаг: %.6f\n", e);
    return integral;
}

// площадь прямоугольного треугольника
float Square(float A, float B)
{
    if (A <= 0.0f || B <= 0.0f)
    {
        printf("[lib2] Ошибка: стороны должны быть положительными\n");
        return 0.0f;
```

```
    }

    printf("[lib2] Фигура: прямоугольный треугольник\n");
    return (A * B) / 2.0f;
}
```

program1.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "../include/contract.h"

void print_help()
{
    printf("\nКоманды:\n");
    printf("  0              - выход из программы\n");
    printf("  1 A B e        - интеграл sin(x) на отрезке [A, B] с шагом e\n");
    printf("  2 A B          - площадь фигуры со сторонами A и B\n");
    printf("\nПримеры:\n");
    printf("  1 0 3.14159 0.001  - интеграл от 0 до pi\n");
    printf("  2 5 10             - площадь фигуры со сторонами 5 и 10\n");
    printf("\n> ");
}

int main()
{
    char input[256];

    printf("========================================\n");
    printf("Программа 1: Статическая линковка\n");
    printf("Используется lib1.so (прямоугольники + прямоугольник)\n");
    printf("========================================\n");

    print_help();

    while (1)
    {
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        // удаляем символ новой строки
        input[strcspn(input, "\n")] = 0;

        if (strlen(input) == 0)
        {
            printf("> ");
            continue;
        }

        if (input[0] == '0')
        {
            printf("Выход из программы.\n");
            break;
        }
        else if (input[0] == '1')
        {
            float A, B, e;
            if (sscanf(input + 2, "%f %f %f", &A, &B, &e) == 3)
            {
                printf("Вычисление интеграла sin(x) от %.4f до %.4f с шагом %.6f...\n", A, B, e);
                float result = SinIntegral(A, B, e);
                printf("Результат: %.8f\n", result);
```

```
            }
            else
            {
                printf("Ошибка: неверные аргументы. Формат: 1 A B e\n");
                printf("Пример: 1 0 3.14159 0.001\n");
            }
        }
        else if (input[0] == '2')
        {
            float A, B;
            if (sscanf(input + 2, "%f %f", &A, &B) == 2)
            {
                printf("Вычисление площади фигуры со сторонами %.4f и %.4f...\n", A, B);
                float result = Square(A, B);
                printf("Результат: %.8f\n", result);
            }
            else
            {
                printf("Ошибка: неверные аргументы. Формат: 2 A B\n");
                printf("Пример: 2 5 10\n");
            }
        }
        else
        {
            printf("Неизвестная команда. Доступные команды: 0, 1, 2\n");
        }

        printf("\n> ");
    }

    return 0;
}
```

program2.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dlfcn.h>

// типы указателей на функции
typedef float (*SinIntegralFunc)(float, float, float);
typedef float (*SquareFunc)(float, float);

int main()
{
    void *library_handle = NULL;
    SinIntegralFunc SinIntegral = NULL;
    SquareFunc Square = NULL;

    int current_library = 1; // 1 = lib1.so, 2 = lib2.so

    printf("Программа 2: Динамическая загрузка библиотек\n");
    printf("Использует интерфейс dlopen/dlsym\n");
    printf("========================================\n");

    // функция для загрузки библиотеки
    int load_library(int lib_number)
    {
        static void *libm_handle = NULL;
        if (!libm_handle)
        {
            libm_handle = dlopen("libm.so.6", RTLD_LAZY | RTLD_GLOBAL);
            if (!libm_handle)
            {
```

```
        fprintf(stderr, "Ошибка загрузки libm: %s\n", dlerror());
        return 0;
    }
}
// закрываем предыдущую библиотеку
if (library_handle != NULL)
{
    dlclose(library_handle);
    library_handle = NULL;
}

// формируем правильное имя файла
char lib_name[20];
if (lib_number == 1)
{
    strcpy(lib_name, "./lib1.so");
}
else if (lib_number == 2)
{
    strcpy(lib_name, "./lib2.so");
}
else
{
    printf("Неверный номер библиотеки\n");
    return 0;
}

printf("Попытка загрузить библиотеку: %s\n", lib_name);

// загружаем библиотеку
library_handle = dlopen(lib_name, RTLD_LAZY);
if (!library_handle)
{
    fprintf(stderr, "Ошибка dlopen: %s\n", dlerror());
    return 0;
}

// сбрасываем ошибки
dlerror();

// загружаем первую функцию
SinIntegral = (SinIntegralFunc)dlsym(library_handle, "SinIntegral");
char *error = dlerror();
if (error != NULL)
{
    fprintf(stderr, "Ошибка dlsym SinIntegral: %s\n", error);
    dlclose(library_handle);
    library_handle = NULL;
    return 0;
}

// загружаем вторую функцию
Square = (SquareFunc)dlsym(library_handle, "Square");
error = dlerror();
if (error != NULL)
{
    fprintf(stderr, "Ошибка dlsym Square: %s\n", error);
    dlclose(library_handle);
    library_handle = NULL;
    return 0;
}

current_library = lib_number;
printf("Успешно загружена библиотека %d\n", lib_number);
if (lib_number == 1)
```

```
    {
        printf("Используется: метод прямоугольников + прямоугольник\n");
    }
    else
    {
        printf("Используется: метод трапеций + треугольник\n");
    }

    return 1;
}

// начальная загрузка библиотеки
if (!load_library(1))
{
    fprintf(stderr, "Не удалось загрузить начальную библиотеку\n");
    return 1;
}

char input[256];
printf("\nКоманды:\n");
printf("  0 - переключить библиотеку\n");
printf("  1 A B e - интеграл sin(x) на отрезке [A,B] с шагом e\n");
printf("  2 A B - площадь фигуры со сторонами A и B\n");
printf("  9 - выход\n");
printf("> ");

while (fgets(input, sizeof(input), stdin))
{
    // удаляем символ новой строки
    input[strcspn(input, "\n")] = 0;

    if (strlen(input) == 0)
    {
        printf("> ");
        continue;
    }

    if (input[0] == '9')
    {
        printf("Выход из программы\n");
        break;
    }
    else if (input[0] == '0')
    {
        // переключение библиотеки
        int new_lib = (current_library == 1) ? 2 : 1;
        if (!load_library(new_lib))
        {
            printf("Не удалось переключить библиотеку\n");
        }
    }
    else if (input[0] == '1')
    {
        if (SinIntegral == NULL)
        {
            printf("Ошибка: функция SinIntegral не загружена\n");
            printf("> ");
            continue;
        }

        float A, B, e;
        if (sscanf(input + 2, "%f %f %f", &A, &B, &e) == 3)
        {
            printf("Вычисление интеграла sin(x) на [%.4f, %.4f] с шагом %.6f...\n", A, B, e);
            float result = SinIntegral(A, B, e);
```

```
            printf("Результат: %.6f\n", result);
        }
        else
        {
            printf("Ошибка: неверные аргументы. Используйте: 1 A B e\n");
        }
    }
    else if (input[0] == '2')
    {
        if (Square == NULL)
        {
            printf("Ошибка: функция Square не загружена\n");
            printf("> ");
            continue;
        }

        float A, B;
        if (sscanf(input + 2, "%f %f", &A, &B) == 2)
        {
            printf("Вычисление площади фигуры со сторонами %.4f и %.4f...\n", A, B);
            float result = Square(A, B);
            printf("Результат: %.6f\n", result);
        }
        else
        {
            printf("Ошибка: неверные аргументы. Используйте: 2 A B\n");
        }
    }
    else
    {
        printf("Неизвестная команда\n");
    }

    printf("> ");
  }

  // закрываем библиотеку перед выходом
  if (library_handle != NULL)
  {
    dlclose(library_handle);
  }

  return 0;
}
```

Пример работы

alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_lab_4/build$ ./program1
==========
Программа 1: Статическая линковка
Используется lib1.so (прямоугольники + прямоугольник)
==========

Команды:
  0 - выход из программы
  1 A B e - интеграл sin(x) на отрезке [A, B] с шагом e
  2 A B - площадь фигуры со сторонами A и B

```
Примеры:
  1 0 3.14159 0.001  - интеграл от 0 до pi
  2 5 10             - площадь фигуры со сторонами 5 и 10

> 1 0 6.28 0.001
Вычисление интеграла sin(x) от 0.0000 до 6.2800 с шагом 0.001000...
[lib1] Метод прямоугольников, шаг: 0.001000
Результат: -0.00013133

> 2 6 3
Вычисление площади фигуры со сторонами 6.0000 и 3.0000...
[lib1] Фигура: прямоугольник
Результат: 18.00000000

> 0
Выход из программы.
alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_lab_4/build$
./program2
Программа 2: Динамическая загрузка библиотек
Использует интерфейс dlopen/dlsym
==========================================
Попытка загрузить библиотеку: ./lib1.so
Успешно загружена библиотека 1
Используется: метод прямоугольников + прямоугольник

Команды:
  0 - переключить библиотеку
  1 A B e - интеграл sin(x) на отрезке [A,B] с шагом e
  2 A B - площадь фигуры со сторонами A и B
  9 - выход
> 1 0 6.28 0.001
Вычисление интеграла sin(x) на [0.0000, 6.2800] с шагом 0.001000...
[lib1] Метод прямоугольников, шаг: 0.001000
Результат: -0.000131
> 2 6 3
Вычисление площади фигуры со сторонами 6.0000 и 3.0000...
[lib1] Фигура: прямоугольник
Результат: 18.000000
> 0
Попытка загрузить библиотеку: ./lib2.so
Успешно загружена библиотека 2
Используется: метод трапеций + треугольник
> 1 0 6.28 0.001
Вычисление интеграла sin(x) на [0.0000, 6.2800] с шагом 0.001000...
```

```
[lib2] Метод трапеций, шаг: 0.001000
Результат: -0.000131
> 2 6 3
Вычисление площади фигуры со сторонами 6.0000 и 3.0000...
[lib2] Фигура: прямоугольный треугольник
Результат: 9.000000
> 9
Выход из программы
alexaryaz@LAPTOP-B7R2JS7R:/mnt/c/StudyProjects/OSlabs/os_lab_4/build$
```

Вывод

В ходе лабораторной работы были разработаны динамические библиотеки для вычисления математических функций и реализованы два механизма их использования: через статическую линковку на этапе компиляции и через динамическую загрузку во время выполнения.

Ключевым аспектом работы стало использование интерфейса dlfcn.h (dlopen, dlsym), что позволило программе загружать исполняемый код библиотек в адресное пространство процесса непосредственно в runtime. Это обеспечило возможность «горячей» замены алгоритмов (переключение между методом прямоугольников и трапеций) без перезапуска приложения.

Сравнение подходов показало, что динамическая загрузка предоставляет значительную гибкость архитектуры, позволяя создавать модульные приложения с поддержкой плагинов. Реализованная программа успешно продемонстрировала изменение точности вычислений при смене библиотеки, подтверждая корректность работы с указателями на функции и управления ресурсами динамических библиотек.